
Unit
À UNIT
RECIFE

ESTRUTURA DE DADOS AVANÇADA



Filas de Prioridade



O que é uma Fila de Prioridade

É uma estrutura de dados onde os elementos são processados com base em sua prioridade, não apenas na ordem de inserção.

Vantagens de uma Fila de Prioridade?

- Permitem gerenciar tarefas com eficiência, garantindo que os elementos mais importantes sejam tratados primeiro.
- São utilizadas em diversos sistemas reais, como gerenciamento de processos em sistemas operacionais e algoritmos de busca eficientes.

O que é uma Heap?

Uma **Heap** é uma estrutura de dados usada para implementar filas de prioridade. Existem dois tipos principais:

- **Min-Heap**: O menor elemento sempre fica no topo (prioridade mais alta).
- **Max-Heap**: O maior elemento fica no topo.

E como funciona uma Heap?

- Em um **Min-Heap**, cada nó pai tem um valor menor que seus filhos.
- Em um **Max-Heap**, o valor do nó pai é sempre maior que seus filhos.
- O Python usa uma **Min-Heap** por padrão, o que significa que elementos com números menores têm maior prioridade.

Onde usamos Heap no dia a dia?

- **Sistemas operacionais**: Agendamento de processos (tarefas críticas primeiro).
- **Redes de computadores**: Controle de pacotes de dados prioritários.
- **Algoritmos de busca em grafos**: Algoritmo de **Dijkstra** para encontrar o caminho mais curto.
- **Jogos e Inteligência Artificial**: Execução de ações mais urgentes primeiro.

Implementação

- Em Python, podemos usar o módulo heapq, que implementa uma Heap (um tipo especial de árvore binária que mantém a ordem de prioridade).
- Ou se quisermos mais controle, podemos criar nossa própria fila de prioridade com uma classe.


Árvores
Binárias de
Busca (BST)

O que é uma Árvore Binária de Busca?

Uma **Árvore Binária de Busca (Binary Search Tree - BST)** é uma estrutura de dados baseada em árvore que:

- Possui **no máximo dois filhos** para cada nó (esquerdo e direito).
- Mantém a seguinte **propriedade de ordenação**:
  - Para um nó N, todos os nós à **esquerda** têm valores **menores** que N.
  - Todos os nós à **direita** têm valores **maiores** que N.

Vantagens uma Árvore Binária de Busca?

- Permite **buscas rápidas** em O(log n) no melhor caso.
- Estrutura eficiente para **inserção e remoção** de elementos.
- Facilita a **organização ordenada** de dados.

Percursos em Árvores BST

Podemos percorrer a árvore de diferentes maneiras:

- **Pré-ordem**: Visita a raiz, depois a subárvore esquerda e a subárvore direita.
- **Em ordem**: Visita a subárvore esquerda, a raiz e depois a subárvore direita (**gera valores ordenados**).
- **Pós-ordem**: Visita a subárvore esquerda, a subárvore direita e por último a raiz.

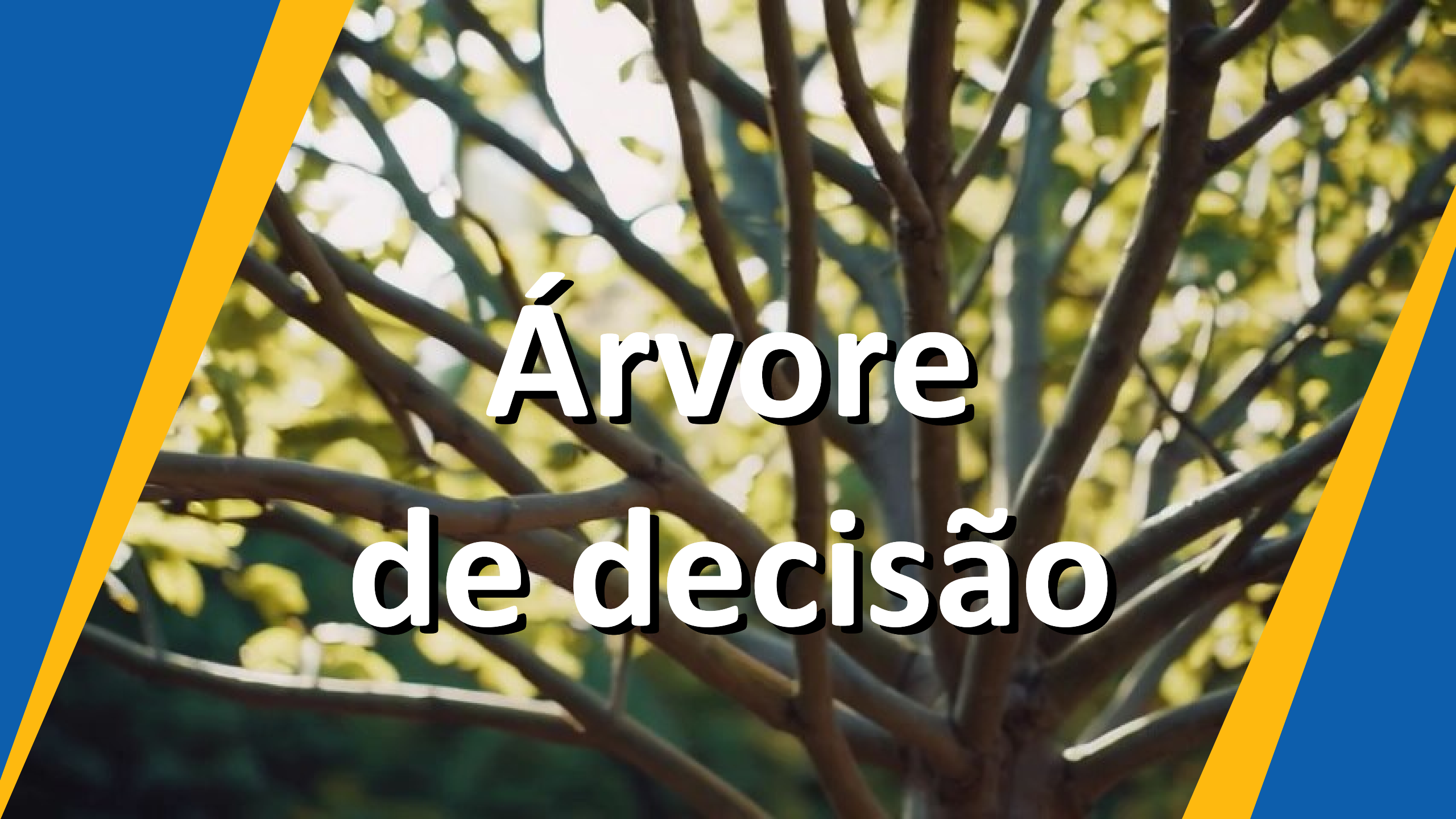
Removendo um Elemento

A remoção de um nó pode ter três casos:

1. **Nó sem filhos**: Apenas removemos.
2. **Nó com um filho**: Substituímos pelo filho.
3. **Nó com dois filhos**: Substituímos pelo menor valor da subárvore direita.

Aplicações das Árvores BST

- Armazenamento eficiente de dados ordenados.
- Indexação em bancos de dados.
- Busca eficiente de informações.
- Implementação de dicionários e sistemas de autocomplete.


Árvore
de decisão

O que é uma árvore de decisão

É um modelo de aprendizado de máquina (Machine Learning) usado tanto para classificação quanto para regressão.

Estrutura da árvore de decisão

Consiste em nós (nodes) que representam testes em atributos (features) do conjunto de dados, ramos (branches) que correspondem aos resultados desses testes e folhas (leaves) que representam as decisões finais (classes ou valores).

Atributos (features)

São as **características** do seu conjunto de dados.

O nó

Em uma árvore de decisão, cada **nó** é como um ponto de verificação onde você faz uma **pergunta** sobre um atributo.

Ramos (branches)

Cada nó (que faz a pergunta) vai ter pelo menos dois **ramos**, cada um representando a **resposta** ou o **resultado** daquele teste.

folhas (leaves)

As **folhas** são os nós **finais** da árvore

folhas (leaves)

Após percorrer vários testes (vários nós e ramos), chegamos a um ponto onde não há mais perguntas a fazer, e temos um **resultado final** (uma **classe** no caso de classificação, ou um **valor** no caso de regressão).

folhas (leaves)

Após percorrer vários testes (vários nós e ramos), chegamos a um ponto onde não há mais perguntas a fazer, e temos um **resultado final** (uma **classe** no caso de classificação, ou um **valor** no caso de regressão).

Se o problema é de classificação, uma folha pode ser: “Esta flor é da espécie Iris Setosa” ou “Este cliente é inadimplente”.

folhas (leaves)

Após percorrer vários testes (vários nós e ramos), chegamos a um ponto onde não há mais perguntas a fazer, e temos um **resultado final** (uma **classe** no caso de classificação, ou um **valor** no caso de regressão).

Se o problema é de regressão, a folha seria um valor numérico predito, por exemplo, “O valor de venda previsto da casa é R$ 350.000”.

Intuição de Funcionamento

Teste de Atributo

- Cada nó da árvore faz uma pergunta do tipo “qual é o valor deste atributo?” ou “o valor desta feature é menor que X?”.

Intuição de Funcionamento

Teste de Atributo

- Cada nó da árvore faz uma pergunta do tipo “qual é o valor deste atributo?” ou “o valor desta feature é menor que X?”.

Divisão dos Dados

- Com base na resposta (por exemplo, “menor ou igual a X” ou “maior que X”), dividimos o conjunto de amostras em subconjuntos.

Intuição de Funcionamento

Teste de Atributo

- Cada nó da árvore faz uma pergunta do tipo “qual é o valor deste atributo?” ou “o valor desta feature é menor que X?”.

Divisão dos Dados

- Com base na resposta (por exemplo, “menor ou igual a X” ou “maior que X”), dividimos o conjunto de amostras em subconjuntos.

Caminho na Árvore

- Repetimos esse processo recursivamente até chegar a um nó folha, que contém uma decisão final (rótulo ou valor predito).

Critérios de Divisão

Impureza de Gini

Critérios de Divisão

Impureza de Gini

- Mede o quão “misturado” está o conjunto de rótulos em um nó. Quanto menor o valor, mais homogêneo é o nó.

Critérios de Divisão

Impureza de Gini

- Mede o quão “misturado” está o conjunto de rótulos em um nó. Quanto menor o valor, mais homogêneo é o nó.

Entropia

Critérios de Divisão

Impureza de Gini

- Mede o quão “misturado” está o conjunto de rótulos em um nó. Quanto menor o valor, mais homogêneo é o nó.

Entropia

- Parecida com a Gini, mede a incerteza dos rótulos. Usada no algoritmo ID3 (um dos primeiros para árvores de decisão).

Critérios de Divisão

Impureza de Gini

- Mede o quão "misturado" está o conjunto de rótulos em um nó. Quanto menor o valor, mais homogêneo é o nó.

Entropia

- Parecida com a Gini, mede a incerteza dos rótulos. Usada no algoritmo ID3 (um dos primeiros para árvores de decisão).

Erro de Classificação

Critérios de Divisão

Impureza de Gini	Entropia	Erro de Classificação
• Mede o quão “misturado” está o conjunto de rótulos em um nó. Quanto menor o valor, mais homogêneo é o nó.	• Parecida com a Gini, mede a incerteza dos rótulos. Usada no algoritmo ID3 (um dos primeiros para árvores de decisão).	• Menos comum, mas também possível (mede a taxa de erros do nó).

O problema de “qual pergunta fazer?”

- Imagine que temos um conjunto de dados de funcionários de uma empresa, e queremos **prever** se um funcionário **vai sair (pedir demissão) ou não**.

Funcionário	Idade	Salário	TempoEmpresa	Cargo	Saiu? (rótulo)
A	25	2000	1	Analista	Não (0)
B	35	4000	3	Desenvolvedor	Sim (1)
C	28	2100	2	Analista	Não (0)
D	45	4500	5	Gerente	Não (0)
E	30	3800	2	Desenvolvedor	Sim (1)

Por que precisamos de uma métrica para escolher a “melhor pergunta”?

Num nó da árvore, podemos perguntar algo como:

- “A **Idade** do funcionário é menor ou igual a 30?”
- “O **Salário** do funcionário é menor ou igual a 3000?”
- “O funcionário tem **Cargo** ‘Desenvolvedor’?”
- etc.

Por que precisamos de uma métrica para escolher a “melhor pergunta”?

- Cada pergunta irá **dividir** (ou **particionar**) nossos funcionários em dois grupos.
- Mas qual pergunta é melhor?
  - É aqui que entram as métricas de impureza (Gini, Entropia) ou o Erro de Classificação.
  - Elas nos dizem quão bem aquela divisão conseguiu separar funcionários que saem dos que não saem.

Exemplo de Impureza de Gini

- A **impureza de Gini** mede quão "misturado" está um conjunto em termos dos rótulos ("Saiu" ou "Não saiu").

$$GINI_{nó} = 1 - \sum_{todas\ as\ classes} p_i^2$$

Onde p_i é a proporção de cada classe naquele nó.

Exemplo de Impureza de Gini

Considere que, depois de uma pergunta, você tem 3 funcionários em um nó:

- 2 “Não saíram”
- 1 “Saiu”

A proporção de “Não saíram” = 2/3 e a de “Saiu” = 1/3.

Quanto **menor** esse valor, **mais puro** o nó.

$$GINI_{nó} = 1 - \left[\left(\frac{2}{3}\right)^2 + \left(\frac{1}{3}\right)^2\right] = \frac{4}{9} \approx 0{,}444$$

- A **Entropia** é outra forma de medir quão misturado o nó está.

$$ENTROPIA_{nó} = 1 - \sum_{todas\ as\ classes} p_i \log_2(p_i)$$

Onde p_i é a proporção de cada classe naquele nó.

Exemplo de Impureza de Entropia

- Usando o mesmo exemplo anterior:

$$ENTROPIA_{nó} = 1 - \left[\frac{2}{3}\log_2\frac{2}{3} + \frac{1}{3}\log_2\frac{1}{3}\right] \approx 0{,}92\ldots$$

o valor é maior que 0, indicando algum grau de desordem.

Exemplo de Impureza de Entropia

Erro de Classificação

- Métrica mais simples e menos usada.
- Basicamente, mede a **taxa de erros** se rotulássemos todos os funcionários do nó como sendo da classe majoritária.

Erro de Classificação

- Se, em um nó, temos 3 funcionários e apenas 1 deles “Saiu”, a classe majoritária é “Não saiu”.
- Então o erro seria quantos não pertencem a essa classe majoritária dividido pelo total, ou seja, 1/3.
- Ela funciona, mas não costuma ser a melhor para guiar divisões, pois Gini ou Entropia dão informação mais sensível sobre a **pureza** do nó.

Escolhendo a “Melhor Pergunta”

- Quando o algoritmo de árvore de decisão vai construir o **primeiro nó**, ele testa todas as possíveis perguntas (por exemplo, “Idade ≤ 30?”, “Salário ≤ 3000?”, “Cargo = Analista?”, etc.), mede a **impureza total antes** e a **impureza total depois** (após dividir os funcionários nos ramos “Sim” e “Não”).
- A pergunta que **mais reduzir** a impureza (ou a entropia) é escolhida para aquele nó.

Escolhendo a “Melhor Pergunta”

Calculamos a impureza do nó inicial (que contém todos os funcionários).

Escolhendo a “Melhor Pergunta”

Calculamos a impureza do nó inicial (que contém todos os funcionários).

Testamos cada pergunta e vemos como o conjunto se divide em dois subgrupos.

Escolhendo a “Melhor Pergunta”

Calculamos a impureza do nó inicial (que contém todos os funcionários).

Testamos cada pergunta e vemos como o conjunto se divide em dois subgrupos.

Para cada subgrupo, calculamos a impureza individual e fazemos a média ponderada pelo tamanho do subgrupo.

Escolhendo a “Melhor Pergunta”

Calculamos a impureza do nó inicial (que contém todos os funcionários).

Testamos cada pergunta e vemos como o conjunto se divide em dois subgrupos.

Para cada subgrupo, calculamos a impureza individual e fazemos a média ponderada pelo tamanho do subgrupo.

A pergunta que resultar na menor média ponderada de impureza (ou seja, a maior “ganho de pureza”) é a melhor escolha.

Escolhendo a “Melhor Pergunta”

Vamos ver um exemplo?

Exemplo

Vamos fazer um teste **hipotético** em cima de uma pergunta simples: “O salário é menor ou igual a R$ 3000?” e ver como ficariam os subgrupos.

Funcionário	Idade	Salário	TempoEmpresa	Cargo	Saiu? (rótulo)
A	25	2000	1	Analista	Não (0)
B	35	4000	3	Desenvolvedor	Sim (1)
C	28	2100	2	Analista	Não (0)
D	45	4500	5	Gerente	Não (0)
E	30	3800	2	Desenvolvedor	Sim (1)

- **Subgrupo 1**: Salário ≤ 3000
  Funcionários: A (2000, Não saiu), C (2100, Não saiu)
- Rótulos: [Não, Não]
- **Subgrupo 2**: Salário > 3000
  Funcionários: B (4000, Sim), D (4500, Não), E (3800, Sim)
  Rótulos: [Sim, Não, Sim]

Passo 1: Calcular a impureza (Gini) do nó original

No nó original, temos 5 funcionários com rótulos: [Não, Sim, Não, Não, Sim].

- “Não” aparece 3 vezes.
- “Sim” aparece 2 vezes.

Proporções:

- p(Não) = 3/5
- p(Sim) = 2/5

$$Gini_{original} = 0.48$$

Passo 2: Calcular a impureza dos subgrupos após a divisão

Subgrupo 1 (A, C):

- Ambos “Não saiu”.
- p(Não) = 2/2 = 1, p(Sim) = 0.

$Gini_{S1} = 0$ (puro)

Subgrupo 2 (B, D, E):

- Rótulos: [Sim, Não, Sim].
- p(Sim) = 2/3, p(Não) = 1/3.

$Gini_{S2} = 0.44$

Passo 3: Calcular a média ponderada da impureza

- Tamanho do Subgrupo 1 = 2 funcionários
- Tamanho do Subgrupo 2 = 3 funcionários
- Total = 5

$$GINI_{Médio} = \left(\frac{2}{5}x0\right) + \left(\frac{3}{5} + \frac{4}{9}\right) \approx 0{,}267$$

Resultado

- A **impureza caiu** de 0,48 (antes de dividir) para cerca de 0,267 após a divisão “Salário ≤ 3000?”.
- Isso é uma **melhora** (redução de impureza).

Resultado

- O algoritmo repetiria esse cálculo para outras possíveis perguntas (por exemplo, “Idade ≤ 30?”, “TempoEmpresa ≤ 2.5?”, “Cargo = Analista?” etc.) e veria qual pergunta produz a **maior redução** na impureza.
- Aquela pergunta seria escolhida como “melhor pergunta” para o primeiro nó.

E que tal praticar?

Vamos trabalhar com as seguintes colunas:

1. **Idade** (contínua)
2. **Salário** (contínuo)
3. **Tempo de Empresa** (contínuo)
4. **Cargo** (categórico)
5. **Saiu?** (rótulo/target): 0 = Não saiu, 1 = Saiu

E que tal praticar?

Funcionário	Idade	Salário	TempoEmpresa	Cargo	Saiu? (rótulo)
A	25	2000	1	Analista	Não (0)
B	35	4000	3	Desenvolvedor	Sim (1)
C	28	2100	2	Analista	Não (0)
D	45	4500	5	Gerente	Não (0)
E	30	3800	2	Desenvolvedor	Sim (1)

E que tal praticar?

Transformar o atributo Cargo em valores numéricos

Para facilitar cálculos, normalmente transformamos *features* categóricas em números. Aqui, faremos algo simples, um **mapa**:

- Analista → 0
- Desenvolvedor → 1
- Gerente → 2

Assim, criaremos nossas variáveis X (atributos) e y (rótulos).

E que tal praticar?

```python
import numpy as np

# Nosso dicionário para mapear o cargo em inteiro
cargo_map = {
    'Analista': 0,
    'Desenvolvedor': 1,
    'Gerente': 2
}

# Tabela de dados
# Formato: [Idade, Salário, TempoEmpresa, Cargo (mapeado)]
X = np.array([
    [25, 2000, 1, cargo_map['Analista']],
    [35, 4000, 3, cargo_map['Desenvolvedor']],
    [28, 2100, 2, cargo_map['Analista']],
    [45, 4500, 5, cargo_map['Gerente']],
    [30, 3800, 2, cargo_map['Desenvolvedor']]
])

# Rótulos (Saiu? 0 = Não, 1 = Sim)
y = np.array([0, 1, 0, 0, 1])
```

Definindo Funções Auxiliares

- Cálculo da Impureza de Gini
- Dividir o conjunto de dados

E que tal praticar?

```python
def gini_impurity(y_subset):
    """
    Calcula a impureza de Gini de um subconjunto y_subset (rótulos).
    """
    classes, counts = np.unique(y_subset, return_counts=True)
    p = counts / len(y_subset)  # proporções de cada classe
    return 1 - np.sum(p ** 2)
```

```python
def split_data(X, y, feature_index, threshold):
    """
    Dada uma feature e um threshold, divide os dados em dois subconjuntos:
    esquerda (<= threshold) e direita (> threshold).
    """
    left_mask = (X[:, feature_index] <= threshold)
    right_mask = (X[:, feature_index] > threshold)

    X_left, y_left = X[left_mask], y[left_mask]
    X_right, y_right = X[right_mask], y[right_mask]

    return X_left, y_left, X_right, y_right
```

Encontrar a Melhor Divisão (best_split)

Precisamos testar várias divisões possíveis (thresholds) para cada feature e escolher aquela que **reduz mais a impureza**.

- Para cada *feature*, pegamos os **valores únicos** em X.
- Para cada valor único, simulamos a divisão, calculamos a **impureza média ponderada** dos dois ramos.
- Salvamos a melhor (menor) impureza encontrada.

E que tal praticar?

```
def find_best_split(X, y):
    """
    Para cada coluna (feature) em X e cada valor possível (threshold),
    testa a divisão e retorna aquela que gera a menor impureza (Gini).

    Retorna:
       best_feature, best_threshold, (X_left, y_left, X_right, y_right)
    """
    n_samples, n_features = X.shape
    if n_samples <= 1:
        return None, None, None, None, None, None

    # Impureza inicial (antes de dividir)
    best_gini = float("inf")
    best_feature, best_threshold = None, None
    best_left_X, best_left_y = None, None
    best_right_X, best_right_y = None, None

    for feature_index in range(n_features):
        # Obter todos os valores únicos desta feature
        values = np.unique(X[:, feature_index])

        for threshold in values:
            # Dividir os dados
            X_left, y_left, X_right, y_right = split_data(X, y, feature_index,
threshold)
```

```
            if len(y_left) == 0 or len(y_right) == 0:
                # Ignora divisões que geram subconjunto vazio
                continue

            # Calcular impureza média ponderada
            gini_left = gini_impurity(y_left)
            gini_right = gini_impurity(y_right)

            w_left = len(y_left) / n_samples
            w_right = len(y_right) / n_samples

            gini_weighted = w_left * gini_left + w_right * gini_right

            # Verifica se essa divisão é melhor
            if gini_weighted < best_gini:
                best_gini = gini_weighted
                best_feature = feature_index
                best_threshold = threshold
                best_left_X, best_left_y = X_left, y_left
                best_right_X, best_right_y = X_right, y_right

    return best_feature, best_threshold, best_left_X, best_left_y, best_right_X,
best_right_y
```

Critérios de Parada

Se todas as amostras do nó tiverem o mesmo rótulo (por exemplo, todos “Não saiu”), não tem porque dividir mais.

- Se não houver divisão que melhore a pureza (por exemplo, se find_best_split retorna None), paramos.
- Podemos definir limites como max_depth ou min_samples_split, mas aqui faremos algo simples, só checando se ainda há ganho em dividir.

Construindo a Árvore

Estrutura do Nó

Vamos representar um nó como um dicionário com as chaves:

- **'feature'**: índice da feature usada para dividir
- **'threshold'**: valor de corte
- **'left'**: subárvore esquerda
- **'right'**: subárvore direita

Se o nó for folha (não dividimos mais), ele será apenas um valor (0 ou 1, no nosso caso).

Construindo a Árvore

```python
def build_tree(X, y, depth=0, max_depth=None):
    """
    Constrói recursivamente uma árvore de decisão usando impureza de
Gini.
    """
    # Se todos os rótulos são iguais, retorna esse rótulo (nó folha)
    if len(np.unique(y)) == 1:
        return y[0]  # classe única

    # Se atingimos a profundidade máxima (opcional)
    if max_depth is not None and depth >= max_depth:
        # Retorna a classe majoritária
        unique, counts = np.unique(y, return_counts=True)
        return unique[np.argmax(counts)]

    # Tentar encontrar a melhor divisão
    feature_index, threshold, X_left, y_left, X_right, y_right =
find_best_split(X, y)
```

```
    # Se não encontrou divisão (ou algo deu errado), retorna a classe
majoritária
    if feature_index is None:
        unique, counts = np.unique(y, return_counts=True)
        return unique[np.argmax(counts)]

    # Construir as sub-árvores
    left_child = build_tree(X_left, y_left, depth+1, max_depth)
    right_child = build_tree(X_right, y_right, depth+1, max_depth)

    # Retorna um "nó interno" (dict) com a informação
    return {
        'feature': feature_index,
        'threshold': threshold,
        'left': left_child,
        'right': right_child
    }
```

Fazendo Previsões

Para **prever** o rótulo de um único funcionário, percorremos a árvore:

- Se o nó for apenas um valor (0 ou 1), retornamos.
- Caso contrário, comparamos o valor do atributo (X[feature]) com threshold e seguimos para a subárvore adequada.

```python
def predict_one(tree, x):
    """ Faz a predição para um único exemplo x, usando a árvore 'tree'."""
    # Se for um nó folha (valor inteiro)
    if not isinstance(tree, dict):
        return tree
    feature = tree['feature']
    threshold = tree['threshold']

    if x[feature] <= threshold:
        return predict_one(tree['left'], x)
    else:
        return predict_one(tree['right'], x)

def predict(tree, X):
    """“ Faz a predição de um conjunto de amostras X (matriz).
    Retorna uma lista ou array de rótulos previstos. """
    y_pred = []
    for i in range(len(X)):
        y_pred.append(predict_one(tree, X[i]))
    return np.array(y_pred)
```

Executando o Exemplo

Para **prever** o rótulo de um único funcionário, percorremos a árvore:

- Se o nó for apenas um valor (0 ou 1), retornamos.
- Caso contrário, comparamos o valor do atributo (X[feature]) com threshold e seguimos para a subárvore adequada.

```
if __name__ == "__main__":
    # Dados X e y já definidos antes...
    # Construir a árvore
    decision_tree = build_tree(X, y, max_depth=None)
    print("Árvore de Decisão (estrutura):")
    print(decision_tree)

    x_teste1 = np.array([29, 2500, 2, cargo_map['Analista']])

    x_teste2 = np.array([40, 4200, 4,
cargo_map['Desenvolvedor']])

    # Empacotar em um array 2D para poder usar a função 'predict'
    X_teste = np.array([x_teste1, x_teste2])
    y_pred = predict(decision_tree, X_teste)

    print("\nPredição para os dois funcionários de teste:",
y_pred)
```


Árvore
Trie

- Uma **árvore Trie** (também conhecida como "prefix tree" ou "digital tree") é uma estrutura de dados de árvore utilizada principalmente para armazenar e buscar palavras em um dicionário ou conjunto de strings.
  - O principal diferencial dessa árvore é que ela armazena caracteres, com cada nível representando um caractere de uma palavra.
- As Tries são bastante eficientes para operações de busca, inserção e prefixo, pois o acesso aos elementos é baseado nos caracteres de prefixo em vez de buscar pelas palavras completas.

Características da Árvore Trie:

Estrutura:

Cada nó da árvore representa um caractere de uma string, e cada caminho desde a raiz até uma folha ou nó interno representa um prefixo de uma ou mais strings.

Busca rápida:

A Trie permite buscar por palavras ou prefixos de forma eficiente, pois a busca segue o caminho de caracteres.

Armazenamento:

Embora ocupe mais memória que outras estruturas de dados, ela permite a busca e inserção rápidas.

Prefixos comuns:

A Trie aproveita o fato de que muitas palavras podem compartilhar prefixos, o que ajuda a reduzir a redundância no armazenamento.

Operações principais:

Operações Pricipais

1. **Inserção**:
   1. Inserir uma palavra na Trie implica ir criando nós para cada caractere da palavra, se não existirem já, e associar a palavra a um nó final.
- **Busca**:
  - Para buscar uma palavra, basta seguir os nós de acordo com os caracteres da palavra.
- **Prefixo**:
  - Verificar se uma palavra com determinado prefixo existe na árvore é uma operação simples, bastando seguir os nós correspondentes ao prefixo e checar se algum nó de fim de palavra está presente.

1. **Classe TrieNode**:
   Representa um nó na árvore, que contém um dicionário de filhos e um indicador de fim de palavra.

2. **Classe Trie**:
   - **Método insert**: Insere uma palavra na árvore.
   - **Método search**: Verifica se uma palavra está na árvore.
   - **Método starts_with**: Verifica se algum prefixo é compartilhado por uma palavra na árvore.

Complexidade:

Inserção e Busca:

O tempo de execução dessas operações é O(n), onde n é o comprimento da palavra a ser inserida ou buscada.

Espaço:

O consumo de memória pode ser alto devido ao armazenamento de muitos nós, especialmente quando há muitas palavras que compartilham prefixos comuns.

Aplicações da Trie

- Autocompletar (Google, editores de texto)
- Correção ortográfica
- Índices em bancos de dados
- Compressão de textos (algoritmos como LZW)
- Filtragem de palavras proibidas (ex.: jogos online)


Antes de
continuar
vamos
revisar

O que são Árvores BST?

É uma árvore especial que segue estas regras:

- **Cada nó** tem no máximo **dois filhos**: o esquerdo e direito.
  - Os **valores menores ficam à esquerda**.
  - Os **valores maiores ficam à direita**.

https://www.cs.usfca.edu/~galles/visualization/



Árvores Balanceadas e Árvores AVL

O que são Árvores Balanceadas?

Uma **árvore balanceada** é uma árvore binária de busca (**BST - Binary Search Tree**) que mantém sua altura relativamente pequena para garantir uma boa eficiência nas operações de busca, inserção e remoção.

Por que balancear?

Se uma BST ficar **desequilibrada**, pode se tornar uma estrutura **linear**, piorando o desempenho de **O(log n)** para **O(n)**. Isso acontece quando os nós são inseridos em ordem crescente ou decrescente.

Propriedade Principal

Para cada nó da árvore AVL:

$$\text{Fator de balanceamento} = \text{altura(subárvoreesquerda)} - \text{altura(subárvore direita)}$$

O fator de balanceamento deve estar sempre em {-1, 0, 1}.

Se for menor que -1 ou maior que 1, a árvore precisa de uma rotação para corrigir o balanceamento.

O que são rotações?

- As rotações são operações que reorganizam os nós da árvore para mantê-la **balanceada**.
- Uma árvore desbalanceada pode se tornar **ineficiente**.
- Servem para:
  - Melhorar a performance da árvore.
  - Manter a propriedade de ABB.
  - Tornar as operações mais rápidas (busca, inserção, remoção).

Quais são as rotações possíveis?

Zig Rotation (Rotação à Direita)

- Acontece quando inserimos valores **na subárvore esquerda da subárvore esquerda**.
- Usamos uma **rotação simples à direita**.

```
def
rotacao_direita(y):
    x = y.esq
    y.esq = x.dir
    x.dir = y
    return x
```

Rotações Zig E Zag

Quais são as rotações possíveis?

Zag Rotation (Rotação à Esquerda)

- Acontece quando inserimos valores **na subárvore direita da subárvore direita**.
- Usamos uma **rotação simples à esquerda**.

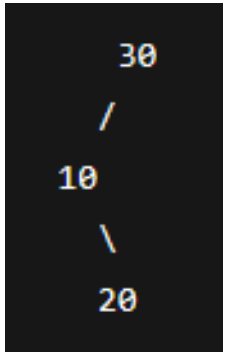
```
def
rotacao_esquerda(x):
    y = x.dir
    x.dir = y.esq
    y.esq = x
    return y
```

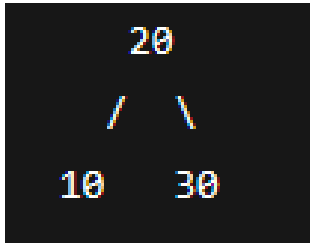
Rotações Zig E Zag

Quais são as rotações possíveis?

Zig-Zag (Esquerda-Direita)

- Caso: Inserção na subárvore direita da subárvore esquerda.
- Correção: Rotação à esquerda no filho, depois rotação à direita no pai.





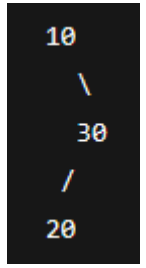

```
def zig_zag(node):
    node.esq =
rotacao_esquerda(node.
esq)
    return
rotacao_direita(node)
```

Rotações Compostas

Quais são as rotações possíveis?

Zag-Zig (Direita-Esquerda)

- Caso: Inserção na subárvore esquerda da subárvore direita.
- Correção: Rotação à direita no filho, depois rotação à esquerda no pai.






```
def zag_zig(node):
    node.dir =
rotacao_direita(node.d
ir)
    return
rotacao_esquerda(node)
```

Rotações Compostas

Árvores AVL

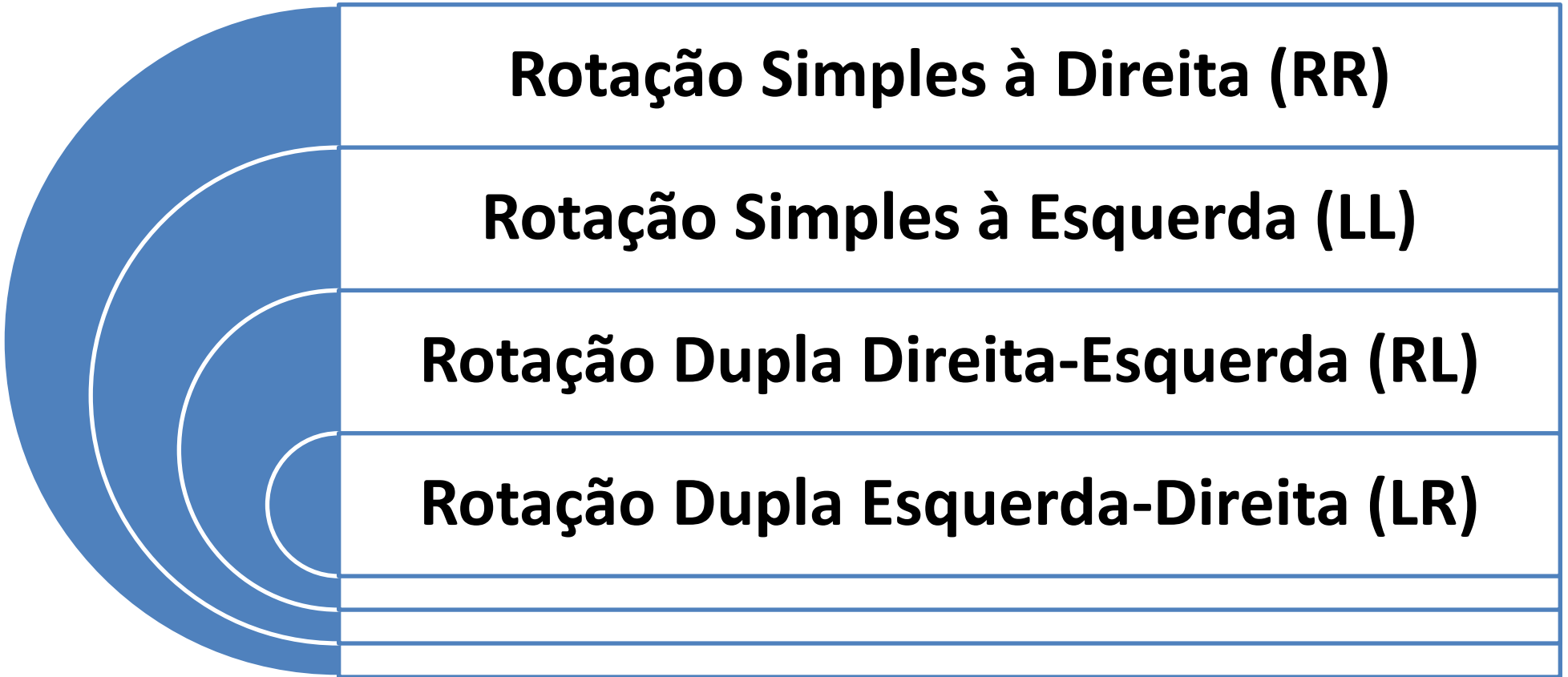
É um tipo específico de árvore binária de busca balanceada, que usa regras matemáticas precisas para se manter equilibrada.

Regras da AVL:

- Cada nó guarda sua altura.
- O fator de balanceamento (altura da esquerda - altura da direita) de cada nó deve ser -1, 0 ou +1.
- Se essa regra for quebrada após uma inserção ou remoção, a árvore se autoajusta com rotações.

Operações na Árvore AVL

Existem 4 tipos de rotação para reequilibrar a árvore:



Árvore AVL

Vantagens	Desvantagens
Melhor para leitura intensiva	Muitas rotações nas inserções/remoções
Altura mais baixa (melhor desempenho)	Estrutura complexa
Mais adequada para dados estáticos	Pouco eficiente para dados mutáveis


Árvores
RED-BLACK

E quando AVL é demais?

A AVL é como um perfeccionista: ela quer tudo 100% balanceado... o tempo todo.

- AVL é excelente, mas:
  - Faz muitas rotações
  - Pode ser mais lenta quando há MUITAS inserções/remoções
- Precisamos de um **balanceamento mais flexível**

O Castelo das Cores

Imagine que cada número que se insere na árvore é um **cavaleiro ou guarda** que quer entrar no castelo.

- **Guardas**  são nós **pretos** – mantêm o castelo seguro.
- **Cavaleiros**  são nós **vermelhos** – podem entrar, mas precisam obedecer regras.

O Castelo das Cores

- **Regras do Castelo (Árvore Red-Black)**
  - A **porta principal (raiz)** do castelo é sempre **um guarda (preto)**.
  - Dois cavaleiros **nunca podem andar juntos** (ou seja, dois vermelhos seguidos são proibidos).
  - Todos os caminhos no castelo até as saídas têm o mesmo número de guardas.
  - Cavaleiros entram primeiro como vermelhos – **o rei vai reorganizar depois** se for necessário.
  - Se houver confusão, o rei **muda as cores ou faz os cavaleiros mudarem de lugar** (rotação ou recoloração).

As 5 Regras da Red-Black Tree

Cada nó é vermelho ou preto

A raiz é sempre preta

Nós vermelhos **não podem ter filhos vermelhos**

Toda folha nula (None) é considerada preta

Todos os caminhos da raiz até uma folha têm o **mesmo número de nós pretos**

As 5 Regras da Red-Black Tree

- Essas regras substituem o "fator de balanceamento" da AVL
- Elas se combinam para impedir que a árvore se desequilibre muito.
- Garantindo que a **altura** de uma Árvore Red-Black seja, no máximo, cerca de $\mathbf{2 \times \log_2(n)}$, o que permite manter as operações em tempo O(log *n*).

Operações na Red-Black Tree

Inserção

- Primeiramente, insere-se o novo nó como se fosse numa **BST comum**.
- Por convenção, o nó inserido é **vermelho** para não impactar a "black-height" de imediato.
- Em seguida, fazemos um procedimento de "fix-up" (ajuste), envolvendo **recolorir** e fazer **rotações** (esquerda ou direita) para restaurar as propriedades acima.

Remoção

- Parecido com a remoção em BST, mas também precisando de um passo extra de **ajuste** para corrigir eventuais violações das propriedades Red-Black.

Rotações

- São operações locais (em torno de um nó) que reestruturam a árvore, mas mantêm a ordem BST. Existem duas: **rotação à esquerda** e **rotação à direita**.
- A rotação desloca nós de forma a manter a propriedade de busca e ajustar as cores para retomar o balanceamento.

Árvore RED-BLACK

Vantagens	Desvantagens
Inserções/remoções mais rápidas	Leitura ligeiramente mais lenta
Menos rotações = mais eficiente	Pode ficar mais desbalanceada que AVL
Usada em muitas bibliotecas padrão	Mais difícil de entender para iniciantes

AVL ou RED-BLACK?

Característica	AVL Tree	Red-Black Tree
Tipo de balanceamento	Estrito	Flexível
Critério de balanceamento	Fator de balanceamento	Cores e black-height
Altura máxima	$\approx 1.44 \log_2(n)$	$\leq 2 \log_2(n)$
Rotações em inserção/remoção	Mais frequentes	Menos frequentes
Inserção	Mais lenta (requer mais rotações)	Mais rápida (recoloração leve)
Remoção	Complexa	Mais simples que AVL
Tempo de busca	Mais rápido (árvore mais rasa)	Um pouco mais lento
Complexidade	Mais difícil de implementar	Mais fácil de implementar


Árvores
SPLAY

Árvore SPLAY

- Uma **árvore Splay** é uma **árvore binária de busca (BST)** que se "autoajusta" toda vez que um nó é acessado.
- Então, quando ao acessar (busca, insere ou remove) um nó, ele é movido para o topo da árvore.

Árvore SPLAY

- Isso torna a árvore mais eficiente para acessos frequentes a certos nós.
- A árvore Splay não é sempre perfeitamente balanceada.
  - Mas é ótima para acessos repetidos.

Mas como funciona o "splay"?

Quando acessamos um nó, realizamos uma série de **rotações** para trazê-lo para a raiz da árvore.

Existem três casos principais:

1. **Zig** – o nó é filho direto da raiz
2. **Zig-Zig** – o nó é filho esquerdo de um filho esquerdo (ou direito de um direito)
3. **Zig-Zag** – o nó é filho direito de um filho esquerdo (ou vice-versa)

Comparando as Árvores

Tipo de árvore	Balanceamento automático?	Acesso rápido a nós repetidos?
BST comum	❌ Não	❌ Não
AVL / Red-Black	✅ Sim	❌ Não
Splay Tree	✅ Sim (parcial)	✅ Sim


Algoritmos
de
ordenação

Vamos rever o que é Heap?

Uma **Heap** é uma estrutura de dados usada para implementar filas de prioridade. Existem dois tipos principais:

- **Min-Heap**: O menor elemento sempre fica no topo (prioridade mais alta).
- **Max-Heap**: O maior elemento fica no topo.

Revisando o Heap

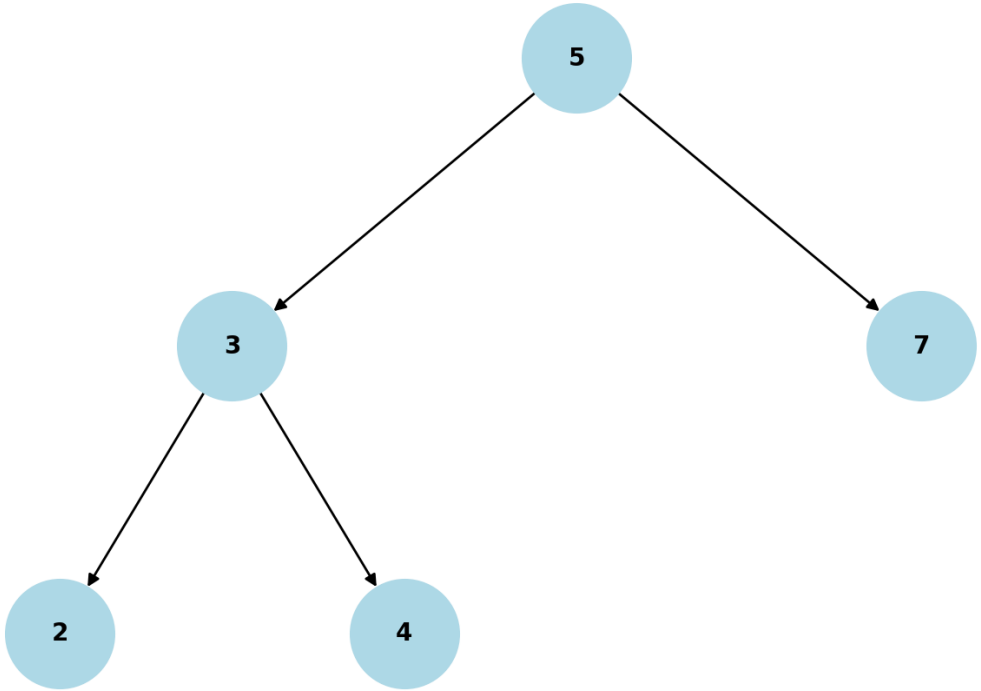
- Em um **Min-Heap**, cada nó pai tem um valor menor que seus filhos.
- Em um **Max-Heap**, o valor do nó pai é sempre maior que seus filhos.
- O Python usa uma **Min-Heap** por padrão, o que significa que elementos com números menores têm maior prioridade.

Cuidado com o HeapSort

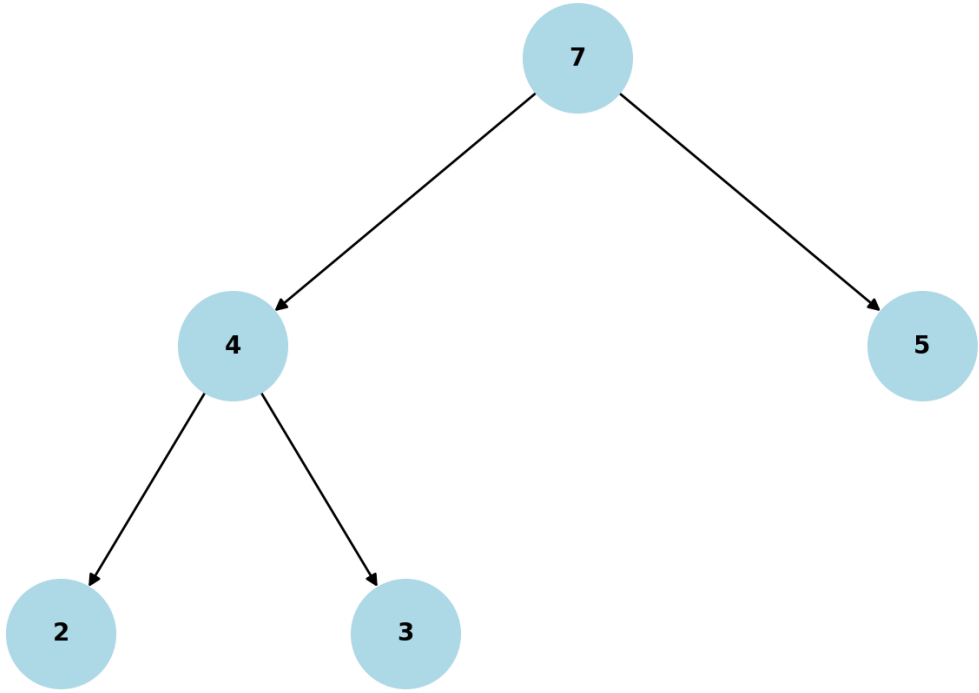
Característica	BST	Heap (Max-Heap)
Ordem dos filhos	Esquerda < pai < Direita	Pai > filhos (ordem dos filhos não importa)
Objetivo principal	Buscar rapidamente	Encontrar rapidamente o maior (ou menor)
Formato da árvore	Pode ser irregular	Sempre completa (preenchida da esquerda para direita)
Inserção mantém ordenação?	Sim, respeita esquerda/direita	Não, só mantém relação pai-filhos
Exemplo clássico de uso	Catálogo ordenado	Fila de prioridade (priority queue)

Cuidado com o HeapSort

BST - Arvore de Busca Binaria



Max-Heap - Estrutura de Prioridade



[5, 3, 7, 2, 4]

Tree Sort

- O Treesort se baseia em árvores binárias de busca (BST).
- Insere elementos na árvore e realiza um percurso in-order.
- Este processo resulta em dados ordenados de maneira eficiente.

TreeSort

- O algoritmo apresenta uma complexidade de O(n log n) no melhor caso.
- No entanto, em cenários de BST desbalanceada, pode chegar a $O(n^2)$.
- A escolha da estrutura da árvore é crucial.

TreeSort

Mas a ideia por trás dele é simples:

- Inserir os elementos na BST.
- Realizar um percurso em ordem (in-order traversal) para obter os elementos ordenados.

TreeSort

Algoritmo	Complexidade Média	Estável	Uso de memória
TreeSort	O(n log n)	Não	O(n)
MergeSort	O(n log n)	Sim	O(n)
QuickSort	O(n log n)	Não	O(log n)
BubbleSort	$O(n^2)$	Sim	O(1)

TreeSort

✅ Vantagens

- Baseado em árvore, ótimo para ensino de estruturas de dados.
- Pode ser adaptado para listas que precisam ser mantidas ordenadas dinamicamente.

⚠️ Desvantagens

- Desempenho ruim se a árvore não for balanceada.
- Mais complexo que outros algoritmos como MergeSort ou QuickSort.

Smoothsort

- Criado por Edsger Dijkstra, é uma variação do Heap Sort
- É mais eficiente para listas quase ordenadas
- Usa **árvores de Leonardo** para otimizar a ordenação
- Melhor caso: O(n), Pior caso: O(n log n)

Números de Leonardo

- Antes de entender a árvore, precisamos conhecer a **sequência de Leonardo**:
  - `L(0) = 1`
  - `L(1) = 1`
  - `L(n) = L(n-1) + L(n-2) + 1`

  `L = 1, 1, 3, 5, 9, 15, 25, ...`
- Cada número representa o **tamanho de uma árvore de Leonardo**.

Árvore de Leonardo

- A **árvore de Leonardo** é uma estrutura especial usada no algoritmo **Smoothsort** para representar grupos de elementos organizados de forma hierárquica (como mini-heaps).
- Ela é baseada nos **números de Leonardo**, que indicam **o número de nós (elementos)** em cada árvore.

Como é uma Árvore de Leonardo?

Uma **árvore de Leonardo de tamanho L(n)** tem **dois filhos**:

- Um filho de tamanho **L(n–1)**
- Um filho de tamanho **L(n–2)**

- Os dois filhos estão **presentes sempre**
- O filho **à esquerda** é sempre **maior que o da direita**

Voltando ao smoothSort

O **Smoothsort** precisa de uma estrutura parecida com um "heap" para ordenar os elementos, mas ele não usa qualquer heap — ele usa uma **floresta de árvores de Leonardo**.

Essas árvores ajudam o algoritmo a:

- ✅ Manter a ordem parcial que já existe na lista.
- ✅ Fazer menos comparações quando os dados já estão quase em ordem.
- ✅ Organizar os dados de forma eficiente para fazer a ordenação.

Floresta de Árvores de Leonardo

- Em vez de uma única grande árvore (como no Heap Sort), o Smoothsort **cria várias árvores menores**, cada uma com tamanho baseado nos **números de Leonardo.**

Floresta de Árvores de Leonardo

Ex: valores = [7, 3, 5, 1, 2, 8, 6, 4, 9, 0]

- O algoritmo constrói uma **floresta**, ou seja, várias árvores.
  O tamanho dessas árvores segue os **números de Leonardo**:
  - 1, 1, 3, 5, 9, 15, 25...
- No caso de 10 elementos, por exemplo, a floresta pode ser composta por árvores de tamanho:
  - [5, 3, 1, 1] ← todos são números de Leonardo

Como o SmoothSort funciona?

1. Constrói a floresta de árvores de Leonardo (montagem das estruturas).
2. Mantém a propriedade de heap dentro de cada árvore (o maior valor está na raiz da árvore).
3. Remove a raiz da maior árvore (como em um heap sort) e reorganiza.
4. Repete o processo até todos os elementos estarem em ordem.

https://www.youtube.com/watch?v=s6ZbB4XxGq8

Comparando

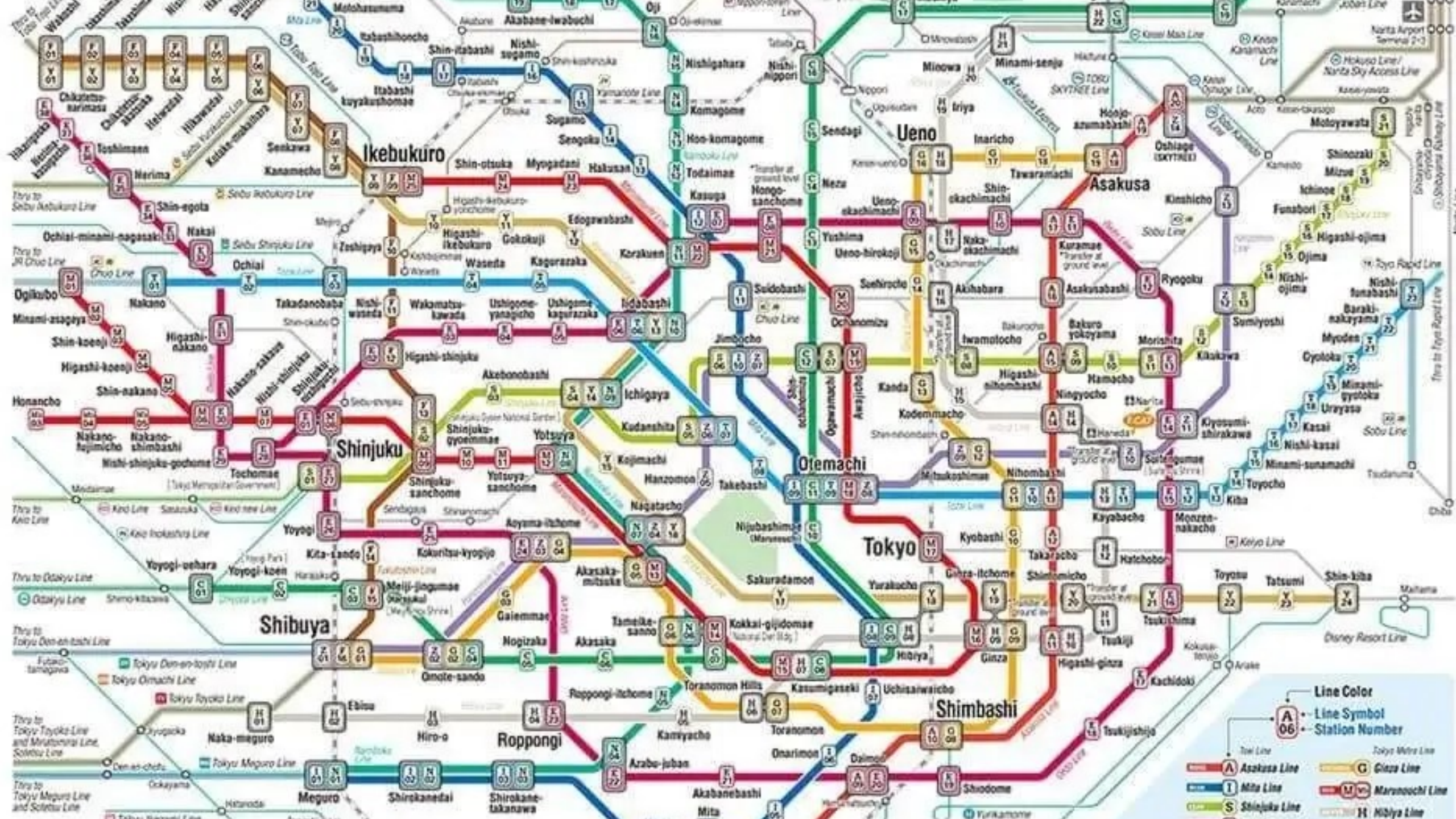
Algoritmo	Estratégia	Complexidade (média)
Bubble Sort	Comparações simples	$O(n^2)$
Quick Sort	Divisão e conquista	O(n log n)
Heap Sort	Heap (árvore binária)	O(n log n)
Smoothsort	Heap + inteligência	O(n) a O(n log n)


Grafos

O que é um grafo?

Imagine um mapa de cidades. As cidades são os pontos (ou nós, ou vértices) e as estradas que ligam essas cidades são as conexões (ou arestas).

Um grafo é uma estrutura que serve justamente para representar coisas que estão conectadas de alguma forma.


Ikebukuro
Shinjuku
Shibuya
Harajuku
Yoyogi-koen
Yoyogi-uehara
Meiji-jingumae
Omote-sando
Ebisu
Naka-meguro
Meguro
Shirokanedai
Shirokane-takanawa
Hiro-o
Roppongi
Azabu-juban
Akabanebashi
Kamiyacho
Toranomon
Toranomon Hills
Kasumigaseki
Hibiya
Tokyo
Otemachi
Nijubashimae
Sakuradamon
Yurakucho
Ginza
Ginza-itchome
Shimbashi
Shiodome
Takebashi
Kudanshita
Ichigaya
Iidabashi
Jimbocho
Suidobashi
Ochanomizu
Ueno
Inaricho
Asakusa
Kuramae
Asakusabashi
Akihabara
Suehirocho
Ueno-hirokoji
Ueno-okachimachi
Naka-okachimachi
Shin-okachimachi
Tawaramachi
Hongo-sanchome
Kasuga
Korakuen
Todaimae
Hon-komagome
Komagome
Nishigahara
Oji
Akabane-iwabuchi
Motohasunuma
Itabashi-honcho
Shin-itabashi
Itabashi-kuyakushomae
Nishi-sugamo
Sugamo
Sengoku
Hakusan
Myogadani
Shin-otsuka
Higashi-ikebukuro
Gokokuji
Edogawabashi
Waseda
Kagurazaka
Ushigome-kagurazaka
Ushigome-yanagicho
Wakamatsu-kawada
Nishi-waseda
Higashi-shinjuku
Shinjuku-nishiguchi
Nishi-shinjuku
Hatsudai
Hashimoto
Keio-hachioji
Takaosanguchi
Keio Line
Keio New Line
Keio Inokashira Line
Sagamihara
Tama Line
Odakyu Line
Odawara
Enoshima
Tokyu Toyoko Line
Tokyu Meguro Line
Tokyu Den-en-toshi Line
Tokyu Oimachi Line
Tokyu Ryoko Line
Tokyu Kodomonokuni Line
Tokyu Setagaya Line
Tokyu Shin-yokohama Line
Yokohama
Minato Mirai Line
Sotetsu Line
Sotetsu Yokohama Line
Sotetsu Izumino Line
Seibu Ikebukuro Line
Seibu Shinjuku Line
Seibu Kokubunji Line
Seibu Haijima Line
Seibu Sayama Line
Seibu Yurakucho Line
Seibu Toshima Line
Seibu Ikebukuro-honcho Line
Seibu Kotesashi Line
Tobu Tojo Line
Tobu Ogose Line
Tobu Kameido Line
Tobu Isesaki Line
Tobu Nikko Line
Tobu Kiryu Line
Tobu Skytree Line
Tobu Urban Park Line
Tobu Daishi Line
Tobu Sakura-shimizu Line
Tobu Koizumi Line
Tobu Hachikoku Line
Tobu Kinugawa Line
Tobu Utsunomiya Line
Tobu Nikko Line
Tobu Kiryu Line
Tobu Isesaki

Componentes:

- Nó (ou vértice): representa um objeto.
- Aresta: ligação entre dois nós.
- Peso (opcional): valor associado à ligação (distância, custo, tempo, etc.).

Aplicações

Onde vemos grafos na vida real:

- Redes sociais (amizades, seguidores)
- Mapas (cidades conectadas por estradas)
- Internet (roteadores conectados por cabos)
- Redes elétricas (postos e linhas de transmissão)
- Biologia (redes neuronais, mapas genéticos)
- Transporte, logística e muito mais.

Formas de Representar Grafos

Matriz de Adjacência é uma **tabela (ou matriz)** onde:

- Cada **linha e coluna representa um nó**.
- Se há conexão entre dois nós, colocamos **1** (ou um peso, se for grafo ponderado).
- Se não há conexão, colocamos **0**.

Formas de Representar Grafos

	A	B	C	D
A	0	1	1	0
B	1	0	0	1
C	1	0	0	1
D	0	1	1	0

Formas de Representar Grafos

Lista de Adjacência é um **dicionário**, onde:

- Cada **chave é um nó**.
- O **valor** é uma **lista dos nós vizinhos** (nós conectados a ele).

```
grafo = {   'A': ['B', 'C'],
            'B': ['A', 'D'],
            'C': ['A', 'D'],
            'D': ['B', 'C']}
```

Formas de Representar Grafos

Lista de Arestas é uma **lista de pares**, onde:

- Cada par representa uma **ligação (aresta)** entre dois nós.

```
arestas = [
    ('A', 'B'),
    ('A', 'C'),
    ('B', 'D'),
    ('C', 'D') ]
```

Tipos de Grafos

1. Direcionado:	2. Não Direcionado:	3. Ponderado:	4. Não Ponderado:
• A conexão tem direção (ex.: seguidor no Twitter).	• Conexão é mútua (ex.: amizade no Facebook).	• As arestas possuem valores(pesos) (distâncias, custos, tempos).	• As arestas não possuem peso. Apenas indicam conexão

Buscas em Grafos

Quando temos um grafo, muitas vezes queremos **percorrer os nós**, encontrar conexões, ou responder perguntas como:

- Existe um caminho de A até B?
- Qual caminho devo seguir?
- Qual é o caminho mais curto (em passos)?
- Para isso, usamos **algoritmos de busca em grafos**.

Busca em Profundidade (DFS – Depth First Search)

Como funciona?

- Imagine que se está explorando um labirinto.
- Se escolhe um caminho e vai o mais longe possível, até não poder mais seguir (beco sem saída).
- Quando não há mais caminho, você volta e tenta outro caminho não explorado.

Funcionamento técnico:

- Funciona como uma pilha (stack) — Último que entra, primeiro que sai (LIFO).
- Bom para percorrer todos os nós ou verificar conexões profundas.
- Implementado em NetworkX pela função dfs_preorder_nodes().
- Vai fundo antes de voltar.

Busca em Largura (BFS – Breadth First Search)

Como funciona?

Imagine que você quer visitar todos os seus amigos mais próximos antes de visitar os amigos dos amigos.

Explora primeiro todos os vizinhos diretos, depois os vizinhos dos vizinhos, e assim por diante.

Funcionamento técnico:

Funciona como uma fila (queue) — Primeiro que entra, primeiro que sai (FIFO).

Vai em camadas (níveis).

Bom para quando se quer encontrar o caminho **mais curto em número de passos**.

Útil em redes sociais (ex.: "amigos de até 2 níveis"), mapas de transporte, jogos e problemas que pedem menor distância.

Menor Caminho – Algoritmo de Dijkstra

Algoritmo que encontra o menor caminho entre um nó de origem e todos os outros nós em um grafo, considerando pesos nas arestas.

- Se o grafo não tem pesos, Dijkstra se comporta como a BFS (encontra o caminho mais curto em quantidade de passos).
- Se o grafo tem pesos, ele encontra o caminho de menor custo total (ex.: menor distância, menor tempo, menor gasto, etc.).

Métodos de busca

Busca	Considera Peso	Garante Menor Caminho
BFS (Largura)	Não	(em passos, sem peso)
DFS (Profundidade)	Não	Não
Dijkstra	Sim	(considerando peso)

Networkx

Biblioteca Python para criação e análise de grafos e redes.

- Suporta grafos simples, direcionados, ponderados e não ponderados.
- Inclui algoritmos como busca (DFS, BFS), menor caminho (Dijkstra), centralidade, etc.
- Permite visualização de grafos com Matplotlib.

Como usar NetworkX?

1. Instale: pip install networkx matplotlib
2. Crie um grafo: G = nx.Graph()
3. Adicione arestas: G.add_edge('A', 'B', weight=5)
4. Use algoritmos: nx.shortest_path(G, source, target, weight='weight')
5. Visualize: nx.draw(G) seguido de plt.show()